

Heart Disease MLOps Assignment

This assignment was initially developed and tested on a local Windows laptop using WSL, a Python virtual environment and Docker Desktop. The implementation was maintained in GitHub and then reproduced in the BITS Prayogshala Lab environment using the committed source code and requirements.txt.

In the Lab, I cloned the repository into the persistent workspace, installed Python dependencies, reran data acquisition, analysis, training, tuning and evaluation, and verified the saved model. We then built the Docker image, tested its prediction endpoint, deployed it to Minikube and connected Prometheus and Grafana.

System Architecture

The system architecture covers the complete workflow from data acquisition to model deployment and monitoring. The UCI dataset is split and processed to train and tune Logistic Regression and Random Forest, while MLflow records parameters, metrics and artifacts. The selected pipeline is evaluated, verified and packaged with FastAPI in Docker, then deployed on Kubernetes. GitHub Actions automates code checks, tests and training, while Prometheus and Grafana monitor API availability, traffic, errors and response time.

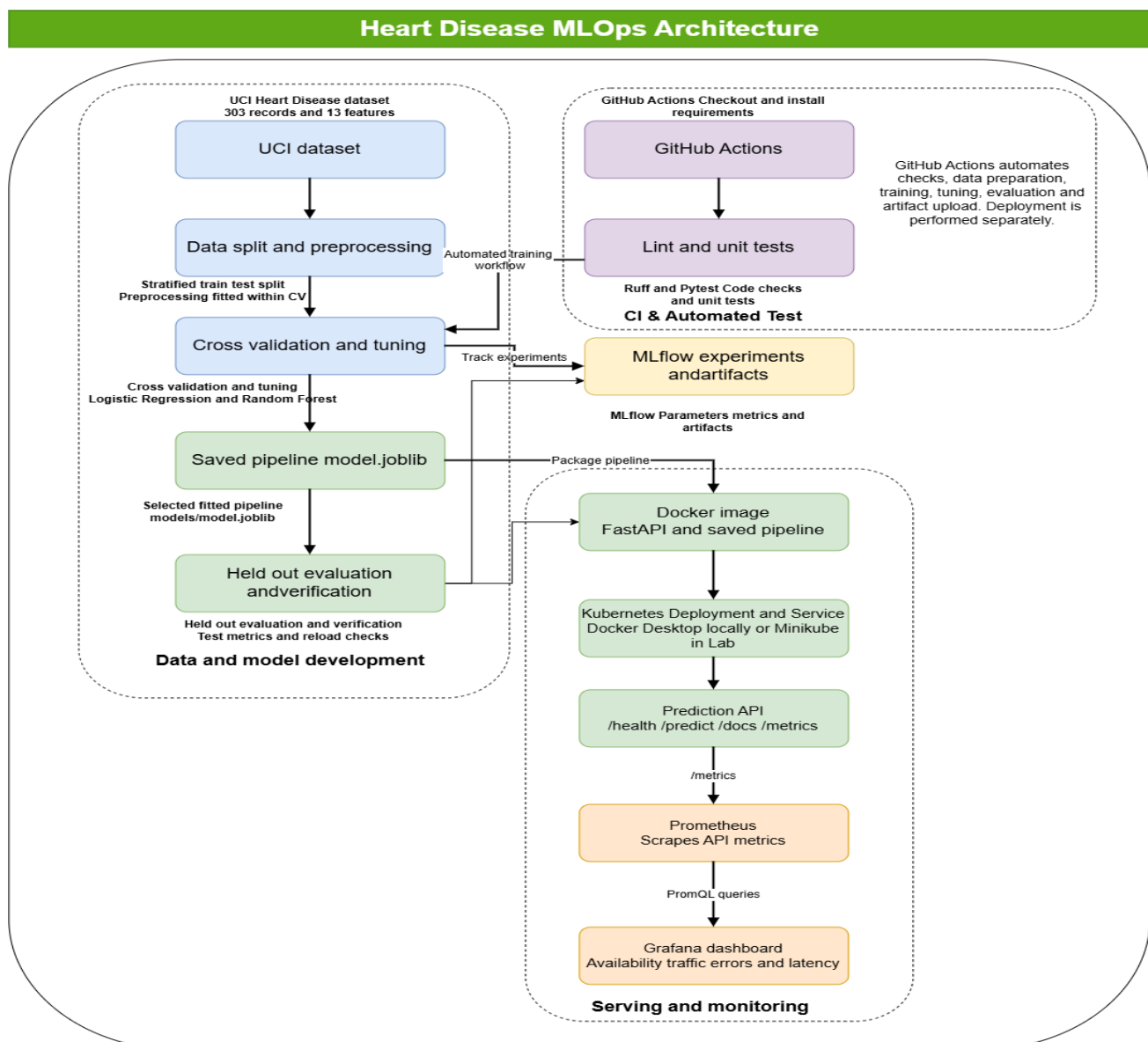


Figure: Heart Disease MLOps Architecture: Data preparation, model training and tracking, automated CI checks, container deployment and API monitoring.

Purpose and result

The project predicts whether heart disease is absent or present from 13 patient features. The final artifact is a fitted preprocessing and Random Forest pipeline saved as models/model.joblib. The Lab execution produced the reported test metrics and demonstrated container serving, Kubernetes deployment and live API monitoring.

Held out test metric	Result
Accuracy	88.52%
Precision	83.87%
Recall	92.86%
ROC AUC	0.9556

Repository and execution location

Repository: <https://github.com/kisangithub12/MLOps>

Lab project directory:

~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops

Reproducing the Assignment in the Lab

The repository provides scripts, dependency file, tests, Dockerfile and deployment configuration. Python 3.12 was used for the workflow. Docker and Minikube are separate platform prerequisites and are not installed by requirements.txt.

Clone and install

```
git clone https://github.com/kisangithub12/MLOps.git
cd MLOps/heart-disease-mlops
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Run the experiment in order

```
python scripts/download_data.py
python scripts/eda.py
python scripts/prepare_data.py
python scripts/train_models.py
python scripts/tune_models.py
python scripts/evaluate_model.py
python scripts/verify_model.py
```

The dataset is downloaded first. Preparation creates the train and test partitions. Baseline training compares the two classifiers, tuning chooses the winning configuration, evaluation uses the held out test records, and verification checks the persisted pipeline. Preprocessing is fitted inside the training pipeline rather than applied globally before splitting.

Run quality checks

```
python -m pytest tests
python -m ruff check scripts tests --select E4,E7,E9,F
```

Step 1 Data Acquisition and Exploration

scripts/download_data.py retrieves the UCI Heart Disease dataset using fetch_ucirepo(id=45). The Cleveland subset contains 303 records, 13 input features and the original num target. The raw CSV is saved to data/raw/heart_disease.csv without imputing values or replacing the original target.

Quality check	Observed result
Missing ca values	4
Missing thal values	2
Duplicate rows	0
Disease absent	164 records 54.1%
Disease present	139 records 45.9%

scripts/eda.py checks types, descriptive statistics, missing values and class balance. Histograms describe continuous feature distributions. The heatmap examines correlations among continuous measurements and the binary target; category codes are excluded from this numerical correlation analysis.

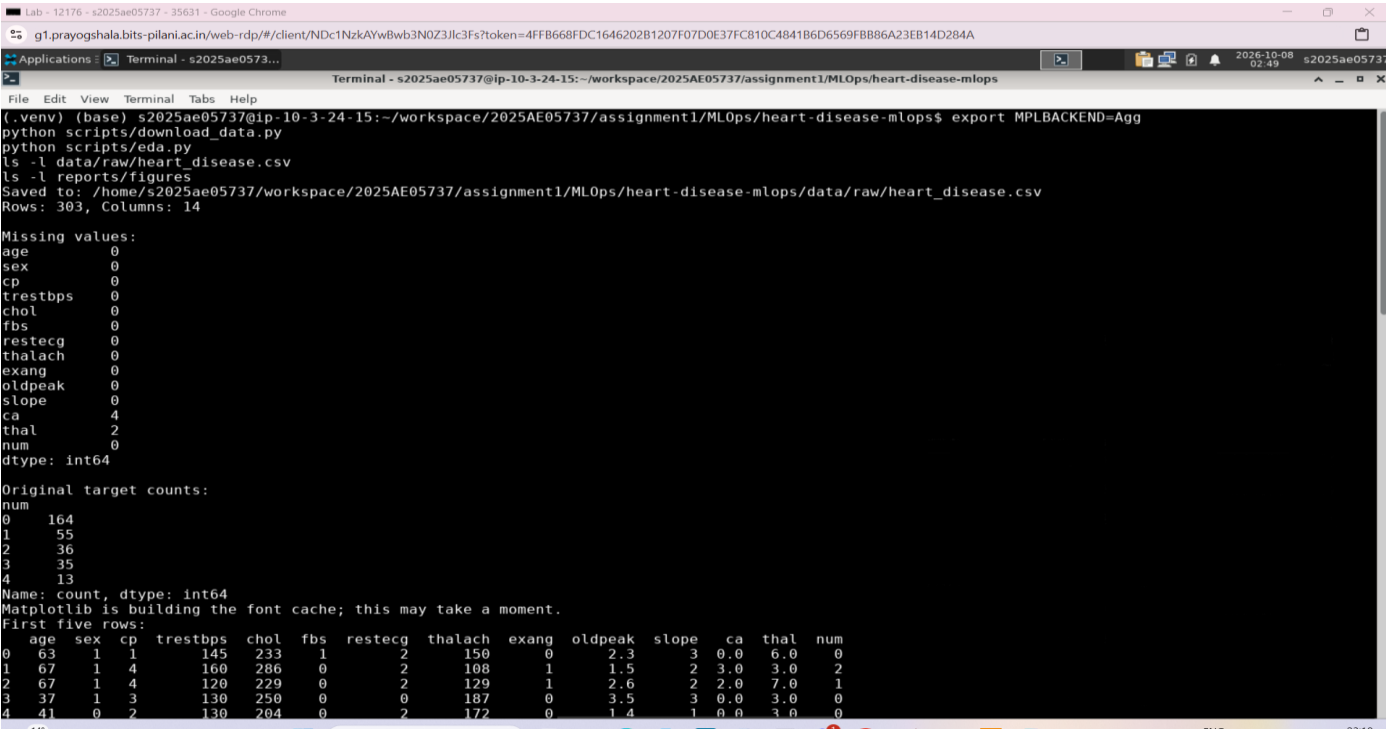


Figure: Lab data acquisition and initial quality checks

```

Lab - 12176 - s2025ae05737 - 35631 - Google Chrome
g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/NDc1NzkAYwBwb3N0Z3Jlc3Fs?token=4FFB668FDC1646202B1207F07D0E37FC810C4841B6D6569FBB86A23EB14D284A

Terminal - s2025ae05737...
File Edit View Terminal Tabs Help

Numerical summary:
      age      sex      cp      trestbps      chol      fbs      ...      exang      oldpeak      slope      ca      thal
num
count 303.000000 303.000000 303.000000 303.000000 303.000000 303.000000 ... 303.000000 303.000000 303.000000 299.000000 301.000000 303.000
mean 54.438944 0.679868 3.158416 131.689769 246.693069 0.148515 ... 0.326733 1.039604 1.600660 0.672241 4.734219 0.937
std 9.038662 0.467299 0.960126 17.599748 51.776918 0.356198 ... 0.469794 1.161075 0.616226 0.937438 1.939706 1.228
min 29.000000 0.000000 1.000000 94.000000 126.000000 0.000000 ... 0.000000 0.000000 1.000000 0.000000 3.000000 0.000
25% 48.000000 0.000000 3.000000 120.000000 211.000000 0.000000 ... 0.000000 0.000000 1.000000 0.000000 3.000000 0.000
50% 56.000000 1.000000 3.000000 130.000000 241.000000 0.000000 ... 0.000000 0.800000 2.000000 0.000000 3.000000 0.000
75% 61.000000 1.000000 4.000000 140.000000 275.000000 0.000000 ... 1.000000 1.600000 2.000000 1.000000 7.000000 2.000
max 77.000000 1.000000 4.000000 200.000000 564.000000 1.000000 ... 1.000000 6.200000 3.000000 3.000000 7.000000 4.000

[8 rows x 14 columns]

Class counts: 0 = absent, 1 = present
num
0 164
1 139
Name: count, dtype: int64

Plot saved to: /home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/reports/figures/class_balance.png
Histograms and correlation heatmap saved.
-rw-r--r-- 1 s2025ae05737 s2025ae05737 12472 Oct 8 02:47 data/raw/heart_disease.csv
total 368
-rw-r--r-- 1 s2025ae05737 s2025ae05737 36760 Oct 8 02:48 class_balance.png
-rw-r--r-- 1 s2025ae05737 s2025ae05737 120856 Oct 8 02:48 correlation_heatmap.png
-rw-r--r-- 1 s2025ae05737 s2025ae05737 103940 Oct 8 02:48 feature_histograms.png
-rw-r--r-- 1 s2025ae05737 s2025ae05737 45837 Oct 8 02:13 test_confusion_matrix.png
-rw-r--r-- 1 s2025ae05737 s2025ae05737 61275 Oct 8 02:13 test_roc_curve.png
(.venv) (base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$

```

Figure: Lab descriptive statistics and saved EDA figures

Step 2.Data Preparation and Baselines

scripts/prepare_data.py converts num into target: 0 for absence and 1 when num is greater than zero. It removes the original target from the inputs and creates a stratified 80 to 20 split with random_state=42: 242 training records and 61 test records.

Feature group	Treatment
age, trestbps, chol, thalach, oldpeak	Median imputation and StandardScaler
sex, cp, fbs, restecg, exang, slope, ca, thal	Most frequent imputation and one hot encoding

The ColumnTransformer and classifier form one pipeline. Each cross validation fold learns its own preprocessing from its training portion, preventing leakage. OneHotEncoder(handle_unknown="ignore") accepts unseen categories. All 13 input features are retained.

Baseline model	Accuracy	Precision	Recall	ROC AUC
Logistic Regression	0.8471	0.8768	0.7834	0.9025
Random Forest	0.8139	0.8129	0.7739	0.8927

Five fold StratifiedKfold with shuffling and seed 42 measures the four metrics. Logistic Regression performed better at baseline. Results are saved in reports/baseline_cv_results.csv.

```

(.venv) (base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ python scripts/prepare_data.py
python scripts/train_models.py
python scripts/tune_models.py
python scripts/evaluate_model.py
Training rows: 242
Test rows: 61

Training class counts:
target
0    131
1     111
Name: count, dtype: int64

Test class counts:
target
0     33
1     28
Name: count, dtype: int64

Saved to: /home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/data/processed
2026/10/08 02:52:01 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/10/08 02:52:01 INFO mlflow.store.db.utils: Updating database tables
Logged baseline run: Logistic Regression
Logged baseline run: Random Forest

Baseline cross-validation results:
      model  accuracy_mean  accuracy_std  precision_mean  precision_std  recall_mean  recall_std  roc_auc_mean  roc_auc_std
Logistic Regression    0.8471         0.0190         0.8768         0.0542         0.7834         0.0459         0.9025         0.0144
Random Forest         0.8139         0.0267         0.8129         0.0258         0.7739         0.0827         0.8927         0.0329

Saved to reports/baseline_cv_results.csv

logistic_regression
Best parameters: {'classifier__C': 0.1, 'classifier__class_weight': 'balanced'}
Best CV ROC-AUC: 0.9028
Logged 8 candidates

random_forest
Best parameters: {'classifier__class_weight': 'balanced', 'classifier__max_depth': 5, 'classifier__min_samples_leaf': 3}
Best CV ROC-AUC: 0.9040
Logged 18 candidates

```

Figure: Lab train test preparation and baseline comparison

Step 2.b Tuning and Final Evaluation

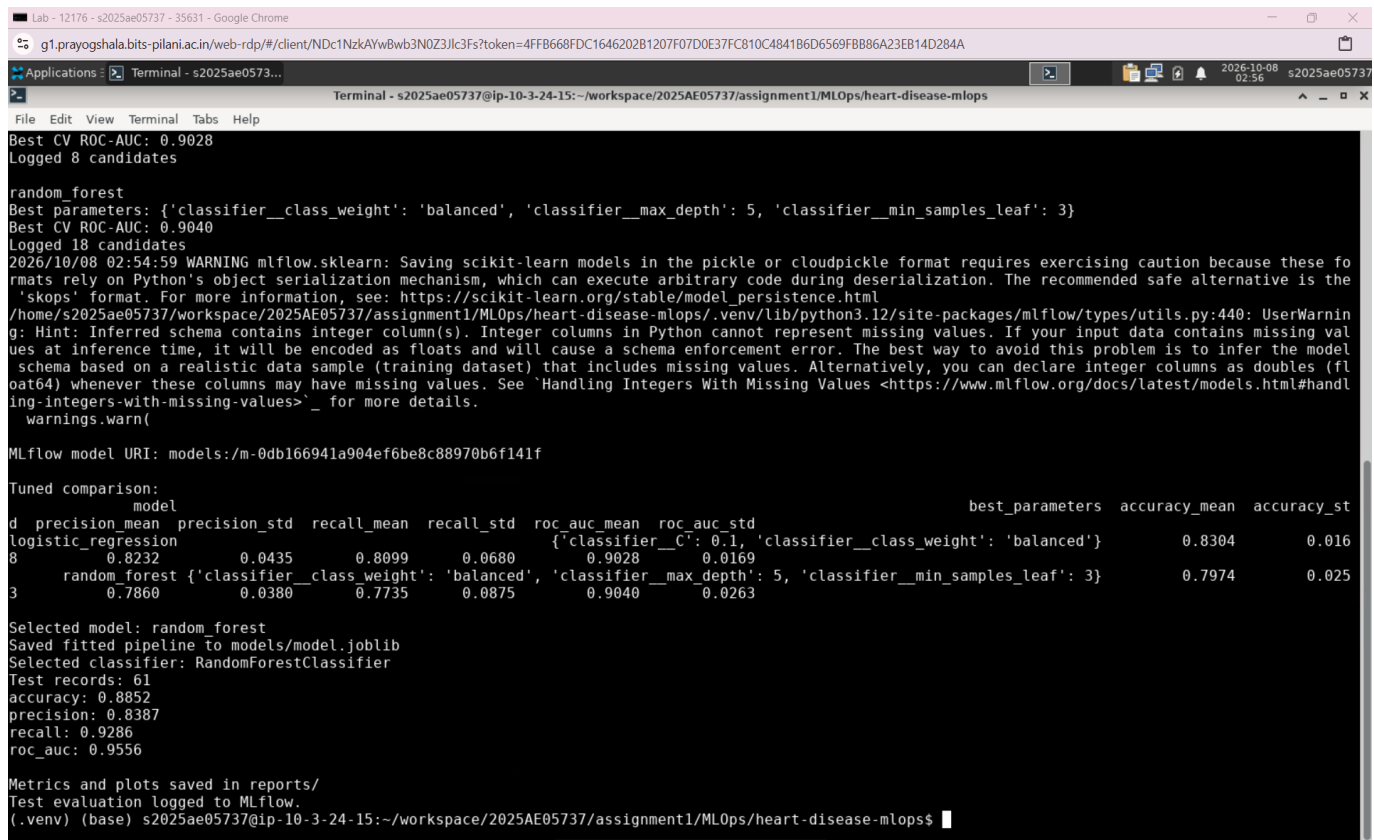
GridSearchCV tests 8 Logistic Regression configurations and 18 Random Forest configurations using five fold cross validation on training data. The predefined selection metric is mean validation ROC AUC.

Tuned model	Best configuration	CV ROC AUC
Logistic Regression	C=0.1; balanced class weights	0.9028
Random Forest	depth=5; leaf size=3; balanced weights	0.9040

Random Forest was selected because it had the highest mean ROC AUC, then refitted on all 242 training records. Its advantage was only about 0.0013; this does not demonstrate clear superiority. Tuned Logistic Regression had higher accuracy, precision and recall at the default threshold.

The test set was not used to select the model. Evaluation on its 61 records gave accuracy 0.8852, precision 0.8387, recall 0.9286 and ROC AUC 0.9556. The confusion matrix contains 26 true positives, 28 true negatives, 5 false positives and 2 false negatives.

Outputs include reports/tuned_cv_results.csv, both complete tuning CSVs, reports/test_metrics.json and the test confusion matrix and ROC curve under reports/figures/.



```

Lab - 12176 - s2025ae05737 - 35631 - Google Chrome
g1.prayogshala.bits-pilani.ac.in/web-rdp/?client/NDc1NzkAYWwBw3N0Z3Jlc3FsToken=4FFB668FDC1646202B1207F07D0E37FC810C4841B6D6569FB86A23EB14D284A

Applications: Terminal - s2025ae05737...
Terminal - s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops

File Edit View Terminal Tabs Help
Best CV ROC-AUC: 0.9028
Logged 8 candidates

random_forest
Best parameters: {'classifier__class_weight': 'balanced', 'classifier__max_depth': 5, 'classifier__min_samples_leaf': 3}
Best CV ROC-AUC: 0.9040
Logged 18 candidates
2026/10/08 02:54:59 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
/home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/.venv/lib/python3.12/site-packages/mlflow/types/utils.py:440: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values' <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>_ for more details.
  warnings.warn(

MLflow model URI: models:/m-0db166941a904ef6be8c88970b6f141f

Tuned comparison:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| model | precision_mean | precision_std | recall_mean | recall_std | roc_auc_mean | roc_auc_std | best_parameters | accuracy_mean | accuracy_std |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| logistic_regression | 0.8232 | 0.0435 | 0.8099 | 0.0680 | 0.9028 | 0.0169 | {'classifier__C': 0.1, 'classifier__class_weight': 'balanced'} | 0.8304 | 0.016 |
| random_forest | 0.7860 | 0.0380 | 0.7735 | 0.0875 | 0.9040 | 0.0263 | {'classifier__class_weight': 'balanced', 'classifier__max_depth': 5, 'classifier__min_samples_leaf': 3} | 0.7974 | 0.025 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

Selected model: random_forest
Saved fitted pipeline To models/model.joblib
Selected classifier: RandomForestClassifier
Test records: 61
accuracy: 0.8852
precision: 0.8387
recall: 0.9286
roc_auc: 0.9556

Metrics and plots saved in reports/
Test evaluation logged to MLflow.
(.venv) (base) s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$

```

Figure: Lab tuned model selection and held out test evaluation

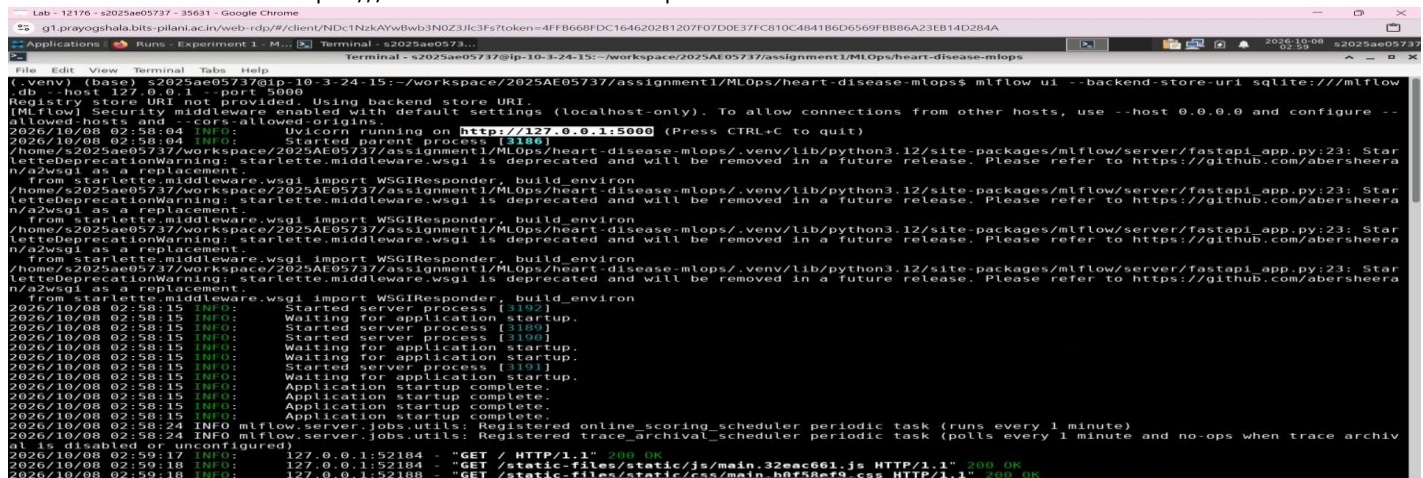
Step 3 MLflow Experiment Tracking

scripts/tracking.py configures the heart-disease-classification experiment. An absolute project based SQLite URI points to mlflow.db and artifacts are stored under mlartifacts/. All training and evaluation scripts use the same tracking setup.

Baseline models have separate runs. Each tuning search has a parent run and nested runs for its candidate configurations. Parameters, fold metrics, means, standard deviations, rankings and comparison CSVs provide a trace of model selection.

The selected model run stores the fitted pipeline, comparison reports, an input example and an MLflow model package. Evaluation logs held out metrics, plots and a SHA 256 model fingerprint. The successful logging implementation uses cloudpickle serialization for the locally trained pipeline.

mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000



```

Lab - 12176 - s2025ae05737 - 35631 - Google Chrome
g1.prayogshala.bits-pilani.ac.in/web-rdp/?client/NDc1NzkAYWwBw3N0Z3Jlc3FsToken=4FFB668FDC1646202B1207F07D0E37FC810C4841B6D6569FB86A23EB14D284A

Applications: Runs - Experiment 1 - MLOps - s2025ae05737...
Terminal - s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops

File Edit View Terminal Tabs Help
(.venv) (base) s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
Registry store URI not provided. Using backend store URI.
[MLflow] Security middleware enabled with default settings (localhost-only). To allow connections from other hosts, use --host 0.0.0.0 and configure --allowed-hosts --cors-allowed-origins.
2026/10/08 02:58:04 INFO: Uvicorn running on http://127.0.0.1:5000 (Press CTRL-C to quit)
2026/10/08 02:58:04 INFO: Started parent process [3188]
/home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/.venv/lib/python3.12/site-packages/mlflow/server/fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated and will be removed in a future release. Please refer to https://github.com/abersheera/n/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_env
/home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/.venv/lib/python3.12/site-packages/mlflow/server/fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated and will be removed in a future release. Please refer to https://github.com/abersheera/n/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_env
/home/s2025ae05737/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops/.venv/lib/python3.12/site-packages/mlflow/server/fastapi_app.py:23: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated and will be removed in a future release. Please refer to https://github.com/abersheera/n/a2wsgi as a replacement.
  from starlette.middleware.wsgi import WSGIResponder, build_env
2026/10/08 02:58:15 INFO: Started server process [3192]
2026/10/08 02:58:15 INFO: Waiting for application startup.
2026/10/08 02:58:15 INFO: Started server process [3193]
2026/10/08 02:58:15 INFO: Waiting for application startup.
2026/10/08 02:58:15 INFO: Started server process [3190]
2026/10/08 02:58:15 INFO: Waiting for application startup.
2026/10/08 02:58:15 INFO: Started server process [3191]
2026/10/08 02:58:15 INFO: Waiting for application startup.
2026/10/08 02:58:15 INFO: Application startup complete.
2026/10/08 02:58:15 INFO: Application startup complete.
2026/10/08 02:58:15 INFO: Application startup complete.
2026/10/08 02:58:15 INFO: Application startup complete.
2026/10/08 02:58:24 INFO mlflow.server.jobs.utils: Registered online_scoring_scheduler periodic task (runs every 1 minute)
2026/10/08 02:58:24 INFO mlflow.server.jobs.utils: Registered trace_archival_scheduler periodic task (polls every 1 minute and no-ops when trace archival is disabled or unconfigured)
2026/10/08 02:59:17 INFO: 127.0.0.1:52184 - "GET / HTTP/1.1" 200 OK
2026/10/08 02:59:18 INFO: 127.0.0.1:52184 - "GET /static-files/static/js/main.32eac661.js HTTP/1.1" 200 OK
2026/10/08 02:59:18 INFO: 127.0.0.1:52188 - "GET /static-files/static/css/main.baf58e79.css HTTP/1.1" 200 OK

```

Figure: MLflow UI launched against the Lab tracking database

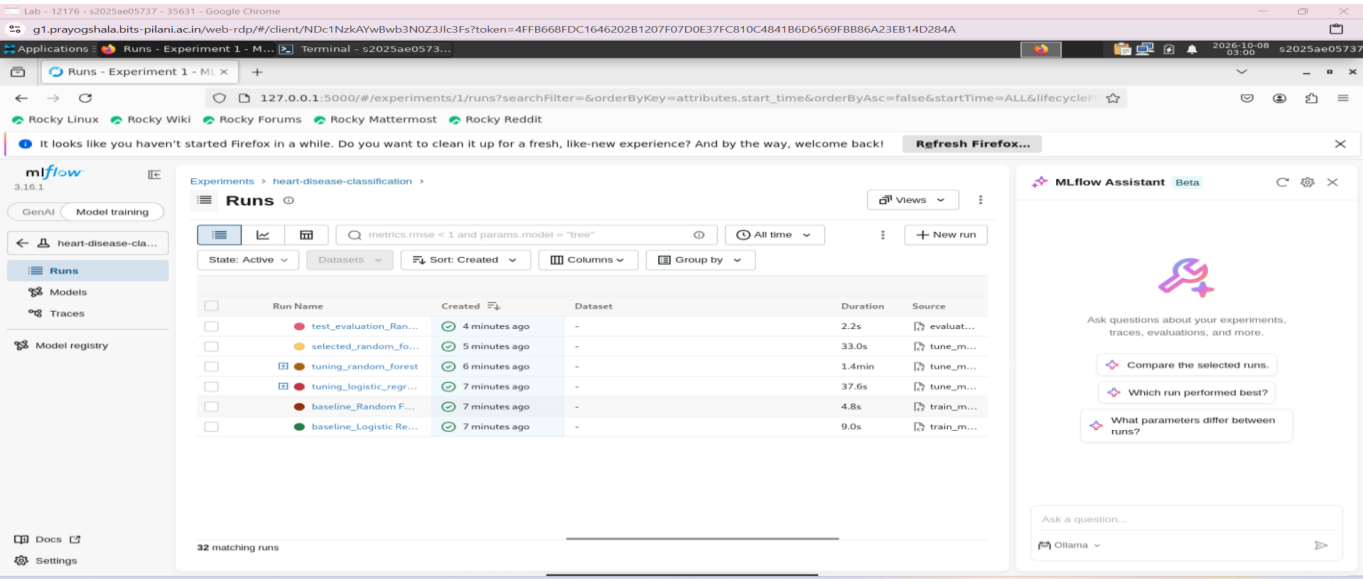


Figure: Experiment runs visible in the Lab MLflow interface

Step 4 Model Packaging and Verification

The final deployment artifact is `models/model.joblib`. It contains the fitted preprocessing steps and `RandomForestClassifier` together, so inference uses the transformations learned during training.

Final Random Forest parameter	Value
n_estimators	200
max_depth	5
min_samples_leaf	3
class_weight	balanced
random_state	42

MLflow also retains the selected pipeline as a tracked model package. The API directly loads the local joblib artifact. The requirements file records dependencies for recreating the Python environment.

Verification performed in the Lab

`scripts/verify_model.py` confirms the pipeline structure, binary predictions for all 61 records, finite probabilities within zero and one, and probability sums of one. Reloaded predictions and probabilities match. A sample containing missing numeric and categorical values is also processed successfully.

`python scripts/verify_model.py`



Figure: Lab verification confirms reload consistency and missing value handling

The Lab training execution complements this persistence check: reloading one artifact verifies inference consistency, while rerunning the scripts in another environment provides reproduction evidence.

Step 5 Automated Tests and Code Checks

Five automated tests validate software behavior independently of the downloaded dataset. Small synthetic inputs make the checks fast and repeatable.

Test area	Behavior checked
Preprocessing 3 tests	Missing values, unseen categories and preservation of fitted training statistics
Model 2 tests	Valid labels and probabilities, and unchanged predictions after saving and reloading

The model tests fit a small Random Forest pipeline rather than depend on an existing final artifact. Inference inputs include missing values, exercising preprocessing and classification together.

```
python -m pytest tests
python -m ruff check scripts tests --select E4,E7,E9,F
```

Ruff checks import placement, problematic statements, syntax issues, undefined variables and unused imports. An earlier misplaced Seaborn import in eda.py was corrected. The Lab run completed with five tests passed and all selected Ruff checks passed.

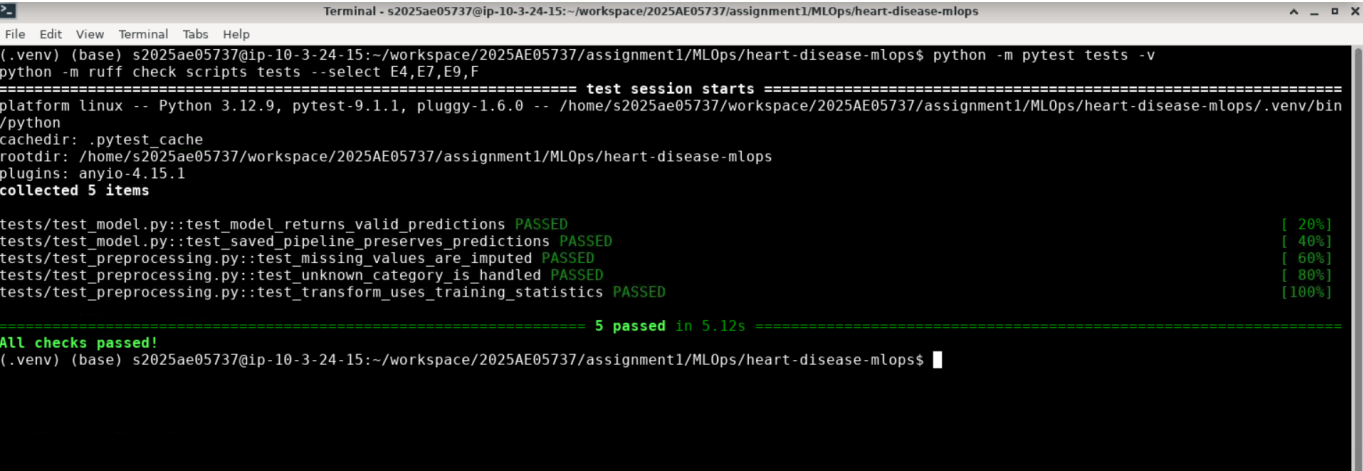


Figure: Five tests passed and Ruff reported all checks passed in the Lab

Meaning of the checks

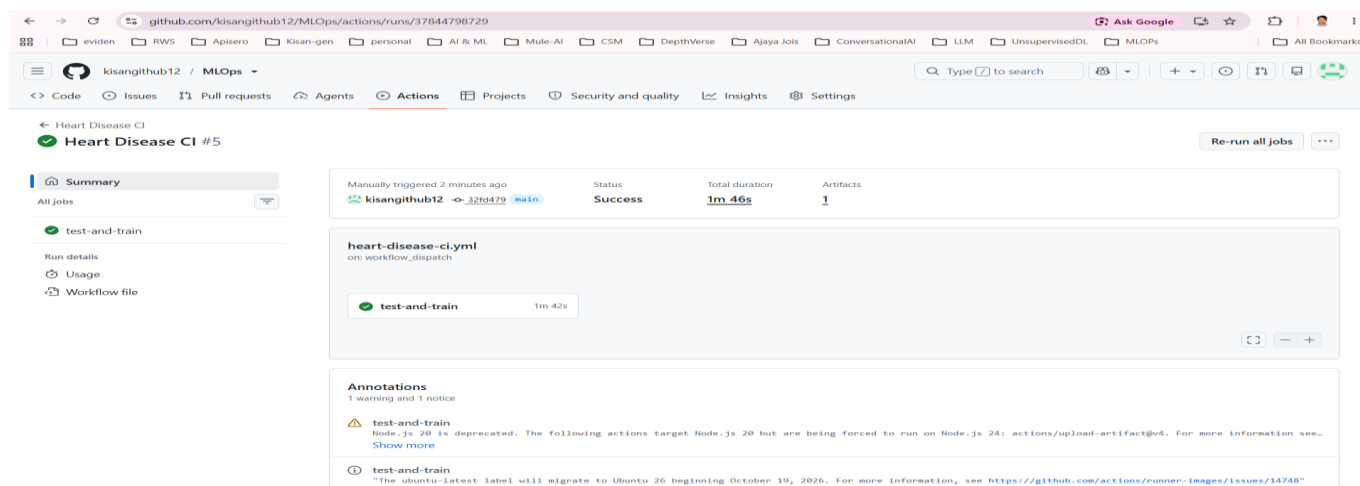
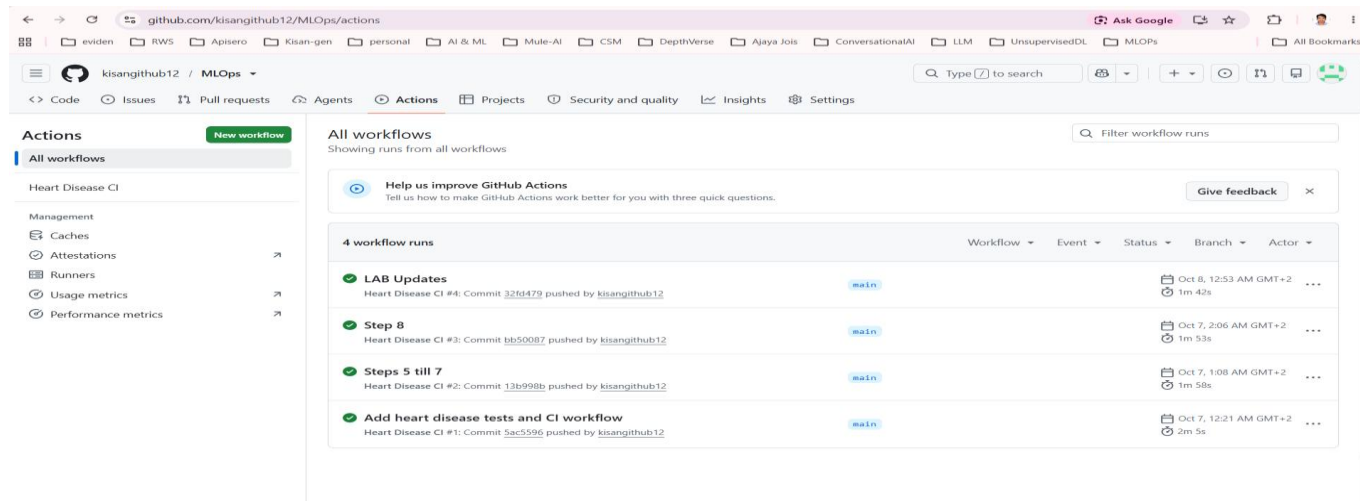
These tests verify implementation and model persistence, not clinical accuracy. Predictive performance is evaluated separately using cross validation and the held out test set.

Step 5.a GitHub Actions and Environment Differences

The repository root workflow `.github/workflows/heart-disease-ci.yml` runs on relevant pushes and pull requests to main, and can be started manually. Path filtering isolates this project from other folders in the MLOps repository.

On a fresh Ubuntu runner, the job checks out code, configures Python 3.12, installs dependencies, runs Ruff and Pytest, downloads and prepares data, trains and tunes models, evaluates the selected model and verifies its artifact. It uploads available reports and models even after a preceding failure.

A failed check stops subsequent normal stages and marks the job failed with logs. The recorded GitHub run succeeded in 2 minutes 5 seconds. CI automates validation and training; Docker and Kubernetes deployment were demonstrated separately. CI MLflow data is separate from the Lab database.



Step 6 Building the Docker Image in the Lab

The Dockerfile uses `python:3.12-slim`, installs `requirements.txt`, copies `app/` and `models/model.joblib`, and starts Uvicorn on container port 8000. The model is included at build time; changing the packaged model requires rebuilding the image.

`.dockerignore` excludes the virtual environment, caches, Git metadata, raw data, reports and MLflow storage. The Lab image uses tag 1.1, including the request logging and metrics instrumentation introduced for monitoring.

```
docker build -t heart-disease-api:1.1 .
docker run -d --name heart-disease-api \
-p 127.0.0.1:8001:8000 heart-disease-api:1.1
```

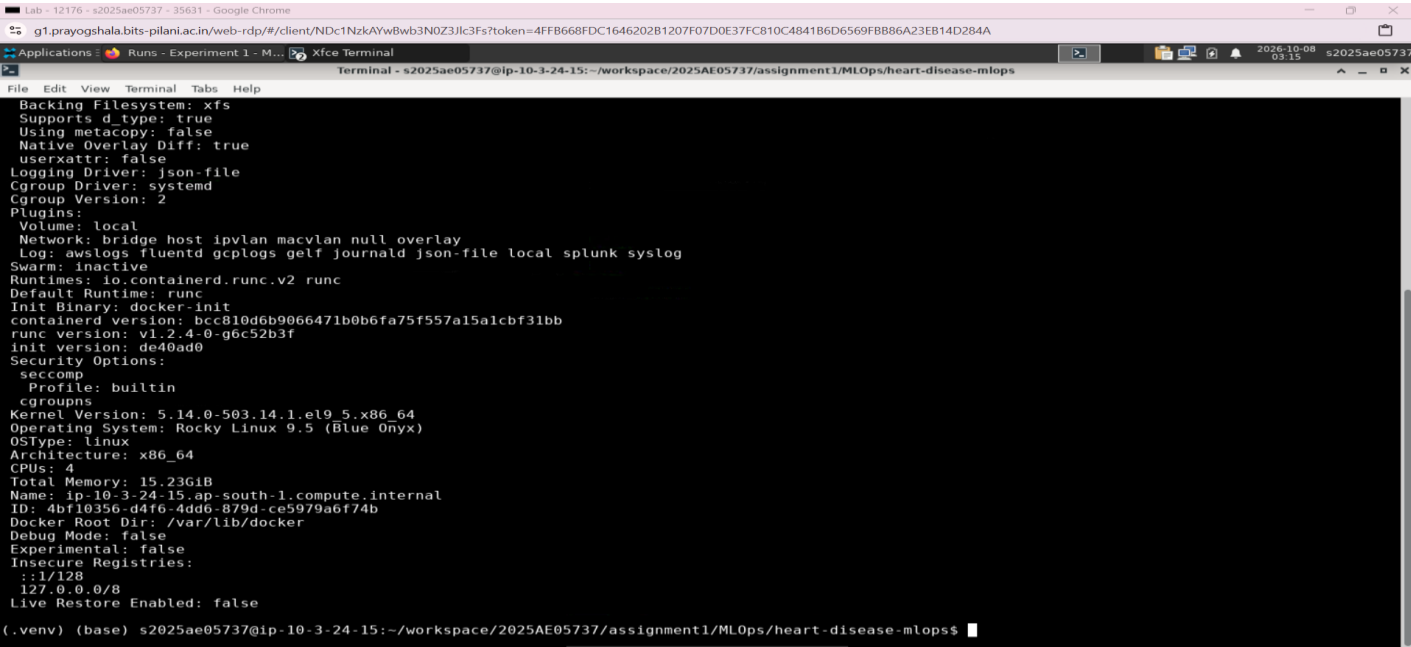


Figure: Docker Engine available in the Lab VM

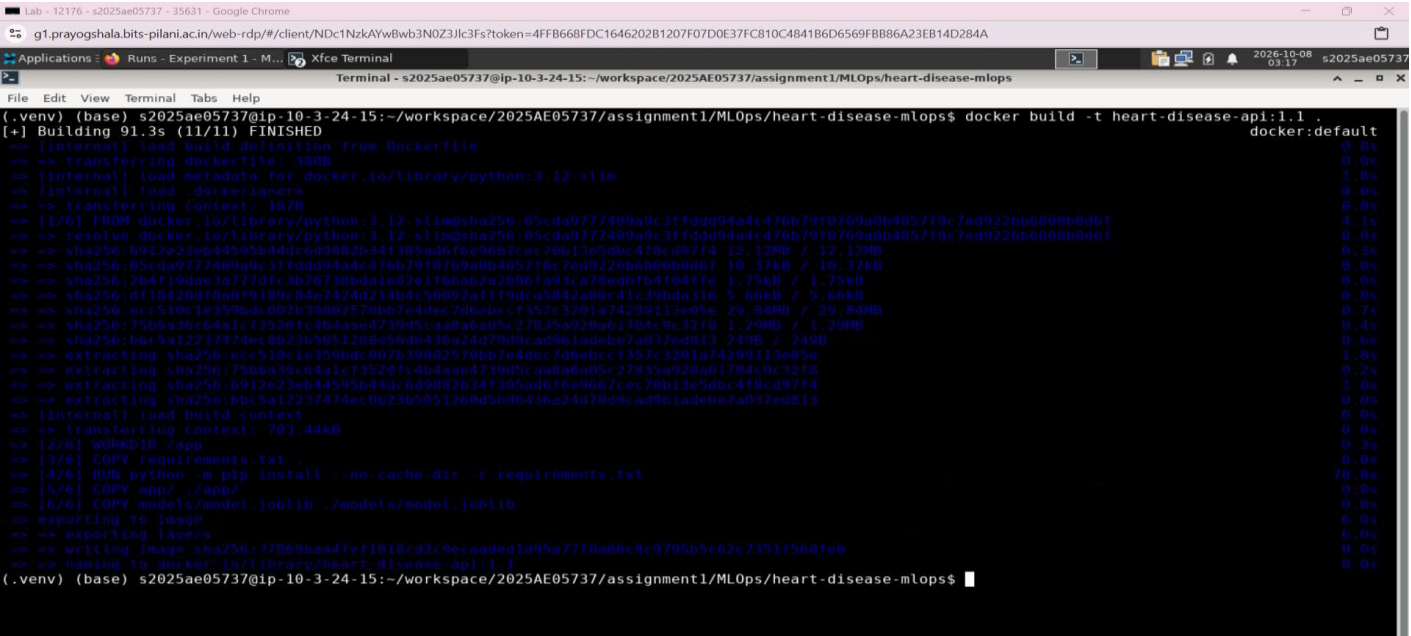


Figure: Successful Docker image build in the Lab

The successful build and the container prediction response on the following page provide isolated serving evidence. This container does not depend on the host Python virtual environment.

Step 6.a Serving Predictions Through FastAPI

app/main.py loads the saved pipeline during startup. The API accepts the original 13 features, constructs the expected model input and returns a prediction without retraining.

Endpoint	Purpose
----------	---------

GET /health	Application health check
POST /predict	Classification and probability response
GET /docs	Interactive API documentation
GET /metrics	Prometheus measurements

prediction is the binary class and label is its readable name. disease_probability is the probability assigned to class 1. confidence is the probability assigned to the predicted class.

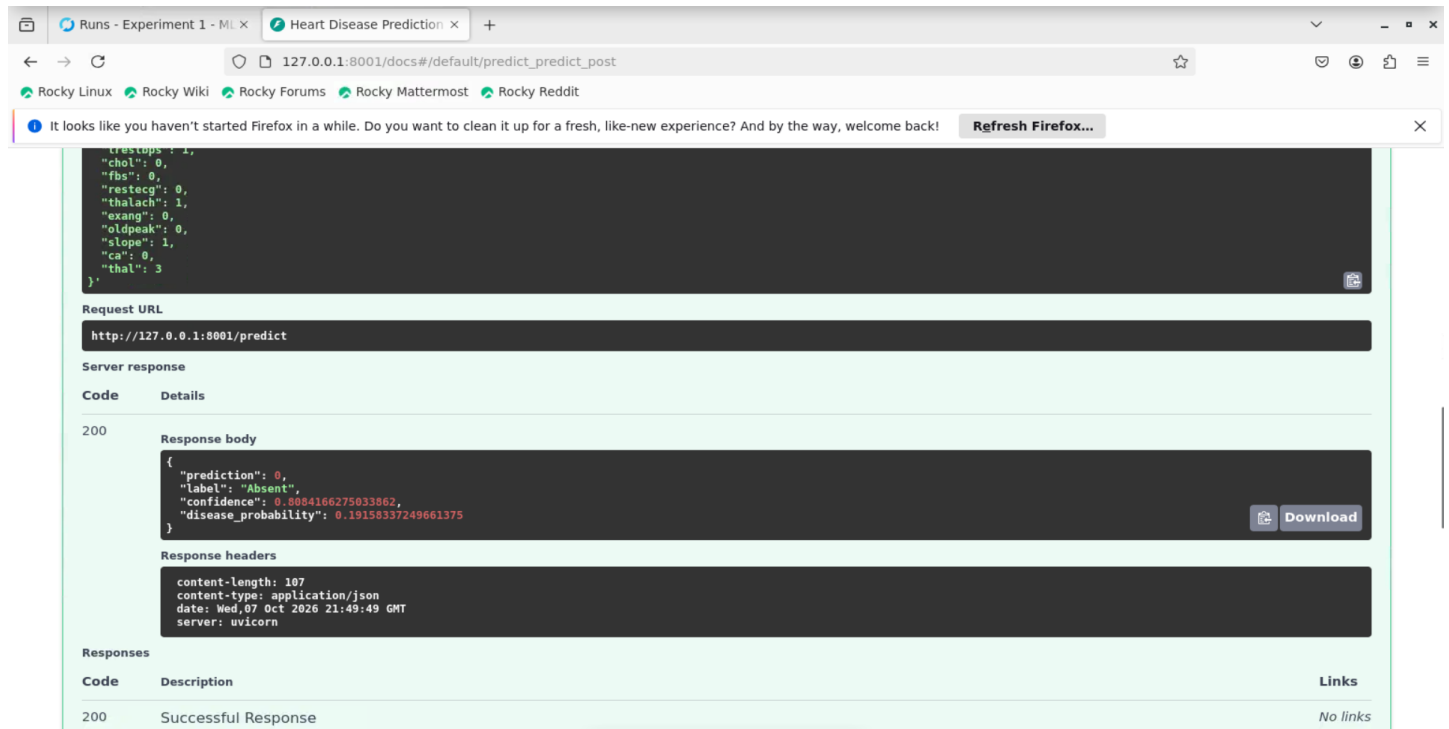


Figure: Successful prediction from the Lab Docker container on port 8001

The demonstrated request returned HTTP 200, class 0 and label Absent, with confidence approximately 0.8084 and disease probability approximately 0.1916. This confirms the packaged API and pipeline served the request successfully.

Step 7 Kubernetes Deployment in the Lab

The initial local deployment used Docker Desktop Kubernetes. For Lab reproduction, Minikube created a local Kubernetes cluster using the Docker driver. Its control plane node reported Ready on Kubernetes v1.32.0.

```
minikube start --driver=docker --cpus=2 --memory=3800
minikube image load heart-disease-api:1.1
minikube kubectl -- apply -f k8s/
minikube kubectl -- rollout status \
  deployment/heart-disease-api --timeout=180s
```

`k8s/deployment.yaml` maintains one replica using the local image with `imagePullPolicy Never`. CPU requests and limits are 100m and 500m; memory requests and limits are 256Mi and 512Mi. Startup, readiness and liveness probes call `/health`.

```

Terminal - s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops
File Edit View Terminal Tabs Help
Using Docker driver with root privileges
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.46 ...
Creating docker container (CPUs=2, Memory=3800MB) ...
Preparing Kubernetes v1.32.0 on Docker 27.4.1 ...
  Generating certificates and keys ...
  Booting up control plane ...
  Configuring RBAC rules ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
(base) s2025ae05737@ip-10-3-24-15:~$ minikube kubectl -- get nodes
NAME      STATUS   ROLES    AGE   VERSION
minikube   Ready    control-plane 90s   v1.32.0
(base) s2025ae05737@ip-10-3-24-15:~$ minikube image load heart-disease-api:1.1
(base) s2025ae05737@ip-10-3-24-15:~$ cd ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops
find . -maxdepth 3 \( -name '*.yaml' -o -name '*.yml' \)
./k8s/service.yaml
./k8s/deployment.yaml
./monitoring/prometheus.yml
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- apply -f k8s/
deployment.apps/heart-disease-api created
service/heart-disease-api created
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- rollout status deployment/heart-disease-api --timeout=180s
deployment "heart-disease-api" successfully rolled out
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- get deployments,pods,services
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api    1/1     1             1           37s

NAME                                READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-5b7dcd967-dpcc5 1/1     Running    0           37s

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api            LoadBalancer  10.100.11.226 <pending>     8002:31266/TCP   37s
service/kubernetes                   ClusterIP     10.96.0.1     <none>         443/TCP           8m28s
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$

```

Figure: Minikube node Ready and deployment successfully rolled out

The bundled minikube kubectl command was used because a standalone kubectl client was unavailable in the Lab. The screenshot shows the initial Service external address pending; the tunnel and final service check are shown next.

Step 7.a Service Access and Prediction Proof

k8s/service.yaml defines a LoadBalancer Service named heart-disease-api. It selects the application pod and maps Service port 8002 to container port 8000.

minikube tunnel

Keep the tunnel running in its own terminal.

After the tunnel started, the Lab Service was accessible at 10.100.11.226:8002. This address is specific to the demonstrated cluster and may change when a new cluster is created. The deployment showed one available replica and the pod was Running.

```

Lab - 12176 - s2025ae05737 - 35631 - Google Chrome
g1.prayogshala.bits-pilani.ac.in/web-rdp/#/client/Ndc1NzkAYwBwb3N0Z3Jlc3Fs?token=4FFB668FDC1646202B1207E07D0E37FC810C4841B6D6569FB886A23EB14D284A
Applications Heart Disease Prediction... Xfce Terminal
Terminal - s2025ae05737@ip-10-3-24-15: ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops
File Edit View Terminal Tabs Help
Creating docker container (CPUs=2, Memory=3800MB) ...
Preparing Kubernetes v1.32.0 on Docker 27.4.1 ...
  Generating certificates and keys ...
  Booting up control plane ...
  Configuring RBAC rules ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
(base) s2025ae05737@ip-10-3-24-15:~$ minikube kubectl -- get nodes
NAME      STATUS   ROLES    AGE   VERSION
minikube   Ready    control-plane 90s   v1.32.0
(base) s2025ae05737@ip-10-3-24-15:~$ minikube image load heart-disease-api:1.1
(base) s2025ae05737@ip-10-3-24-15:~$ cd ~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops
find . -maxdepth 3 \( -name '*.yaml' -o -name '*.yml' \)
./k8s/service.yaml
./k8s/deployment.yaml
./monitoring/prometheus.yml
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- apply -f k8s/
deployment.apps/heart-disease-api created
service/heart-disease-api created
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- rollout status deployment/heart-disease-api --timeout=180s
deployment "heart-disease-api" successfully rolled out
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- get deployments,pods,services
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api    1/1     1             1           37s

NAME                                READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-5b7dcd967-dpcc5 1/1     Running    0           37s

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api            LoadBalancer  10.100.11.226 <pending>     8002:31266/TCP   37s
service/kubernetes                   ClusterIP     10.96.0.1     <none>         443/TCP           8m28s
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$ minikube kubectl -- get service heart-disease-api
NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
heart-disease-api                   LoadBalancer  10.100.11.226 10.100.11.226 8002:31266/TCP   2m42s
(base) s2025ae05737@ip-10-3-24-15:~/workspace/2025AE05737/assignment1/MLOps/heart-disease-mlops$

```

Figure: Available deployment and assigned LoadBalancer service address

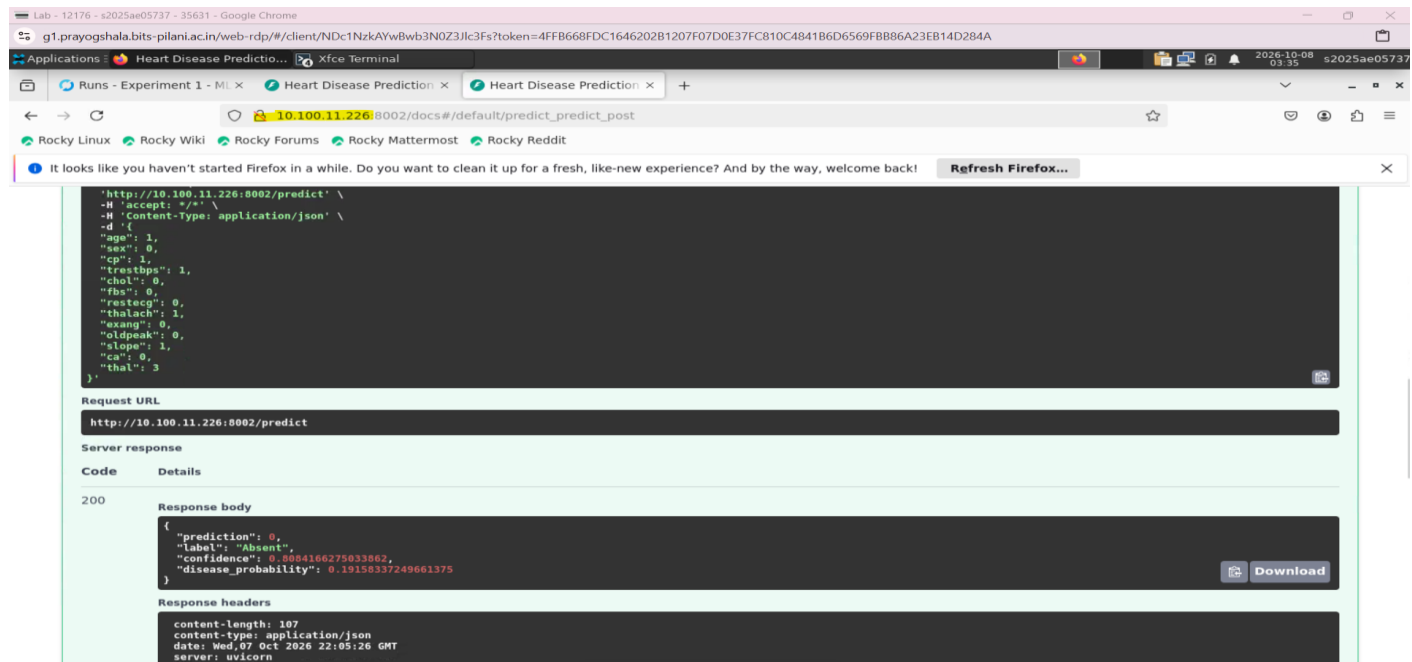


Figure: HTTP 200 prediction served through the Lab Kubernetes Service

The Service prediction confirms that Kubernetes served the packaged pipeline. Health probes check application responsiveness; they do not measure model quality.

Step 8 Request Metrics and Prometheus

FastAPI middleware records request method, path, status and duration without logging patient payloads. `prometheus-fastapi-instrumentator` exposes HTTP counters and duration measurements through `/metrics`.

The Lab configuration `monitoring/prometheus-lab.yml` points to the reachable Kubernetes Service at `10.100.11.226:8002`, replacing the local Docker Desktop target `host.docker.internal:8002`. Prometheus scrapes every 15 seconds.

Prometheus runs with host networking in the Lab so it can reach the Minikube tunnel route. Its UI listens on `127.0.0.1:9090`. Grafana also uses host networking and connects to `http://127.0.0.1:9090`; the local setup used container network naming.



Figure: Original target configuration inspected before Lab adaptation

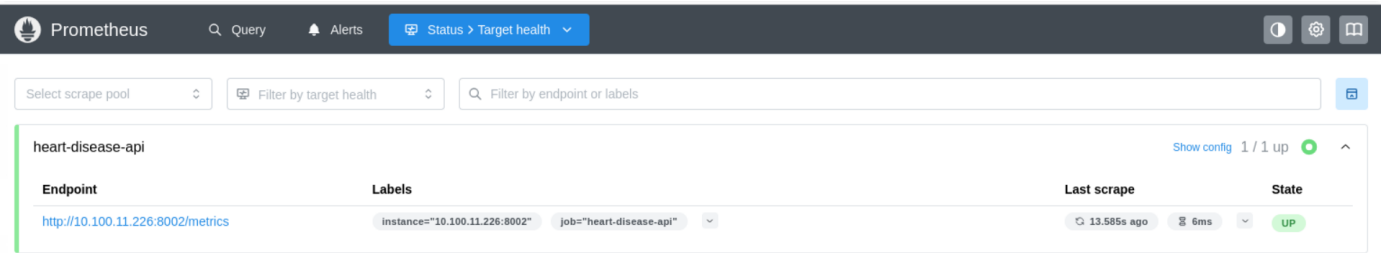


Figure: Prometheus confirms the heart disease API target is UP

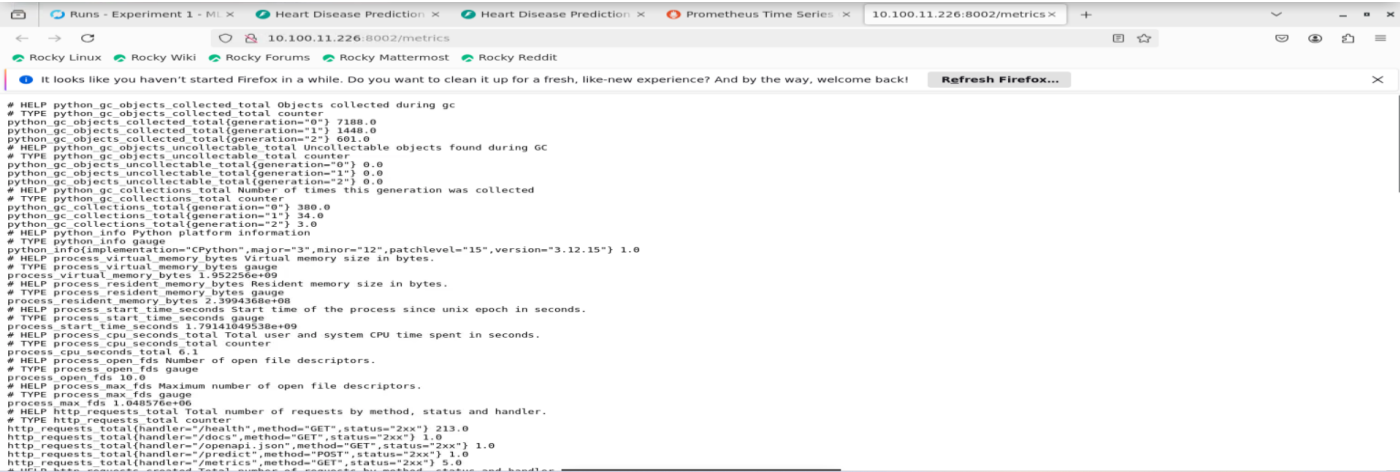


Figure: API metrics exposed through the Kubernetes Service

The UP state proves the latest scrape succeeded. Request logs support individual call investigation, while aggregated metrics support operational observation.

Step 8.a Grafana and Production Readiness

Grafana displays API metrics from Prometheus. The supplied Lab screenshot shows active statistics and a recent response time point. It captures the dashboard in edit mode, with availability and request count values grouped in one panel.

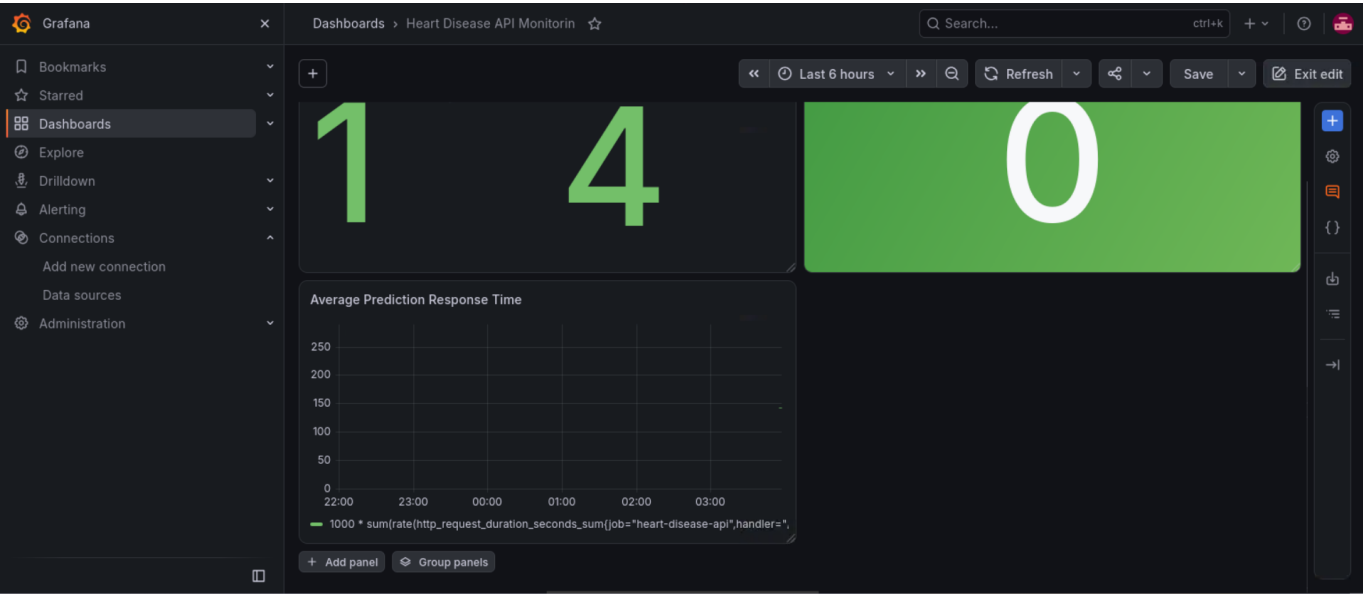


Figure: Lab Grafana dashboard receiving API metrics

Availability uses up{job="heart-disease-api"}. Request and error counters filter handler="/predict"; errors include HTTP 4xx and 5xx. Average duration is calculated from five minute rates of duration sum and count, multiplied by 1000 for milliseconds. It may show no data without recent requests.

Production readiness evidence

Requirement	Evidence in this report
Clean setup using requirements	Lab installation and execution sequence on page 1 &2 ; model results on pages 3–7
Isolated Docker build and serving	Successful Lab build and HTTP 200 response on pages 10–11
Failure on code or test errors with logs	Pytest and Ruff checks on page 8; GitHub Actions failure behavior on page 9

The workflow covers all eight assignment stages. The final model, reports, tests, CI workflow, Dockerfile and Kubernetes manifests provide a reproducible handover. Monitoring observes API availability, traffic, errors and duration.